DATA STRUCTURES ASSIGNMENT 04

1)Implement a stack using queue data structure:

CODE:

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push(int value)
{
   if (top == MAX - 1)
   {
      printf("Stack Overflow\n");
   }
   else
   {
      top++;
      stack[top] = value;
      printf("%d pushed into stack\n", value);
   }
}

void pop()
{
   if (top == -1)
   {
```

```
        printf("Stack Underflow\n");
    }
    else
    {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}

void display()
{
    int i;

    if (top == -1)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Stack elements are:\n");

        for (i = top; i >= 0; i--)
        {
            printf("%d\n", stack[i]);
        }
    }
}
```

```
int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- STACK MENU ---\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
```

```
            display();
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```

OUTPUT:

```
--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter value: 24
24 pushed into stack

--- STACK MENU ---
1. Push
2. Pop
3. Display
4. Exit
```

Enter your choice: 1

Enter value: 23

23 pushed into stack

--- STACK MENU ---

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 3

Stack elements are:

23

24

--- STACK MENU ---

1. Push

2. Pop

3. Display

4. Exit

Enter your choice: 4

Exit

2)implement a queue using stack data structure:

CODE:

```
#include <stdio.h>

#define MAX 5
```

```
int stack1[MAX], stack2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int value)
{
   if (top1 == MAX - 1)
   {
      printf("Queue Overflow\n");
   }
   else
   {
      top1++;
      stack1[top1] = value;
      printf("%d inserted into queue\n", value);
   }
}

void dequeue()
{
   int i;

   if (top1 == -1 && top2 == -1)
   {
      printf("Queue Underflow\n");
      return;
   }

   if (top2 == -1)
```

```
    {
        while (top1 != -1)
        {
            top2++;
            stack2[top2] = stack1[top1];
            top1--;
        }
    }

    printf("%d deleted from queue\n", stack2[top2]);
    top2--;
}

void display()
{
    int i;

    if (top1 == -1 && top2 == -1)
    {
        printf("Queue is empty\n");
        return;
    }

    printf("Queue elements are:\n");

    /* Elements already in stack2 */
    for (i = top2; i >= 0; i--)
    {
```

```
        printf("%d ", stack2[i]);
    }

    /* Elements still in stack1 */
    for (i = 0; i <= top1; i++)
    {
        printf("%d ", stack1[i]);
    }

    printf("\n");
}

int main()
{
    int choice, value;

    while (1)
    {
        printf("\n--- QUEUE USING STACK ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
```

```
        {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
 --- QUEUE USING STACK ---
1. Enqueue
```

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value: 23

23 inserted into queue

--- QUEUE USING STACK ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value: 55

55 inserted into queue

--- QUEUE USING STACK ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 2

23 deleted from queue

--- QUEUE USING STACK ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter value: 45

45 inserted into queue

--- QUEUE USING STACK ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 4

3)write a program using stack data structure to implement managing the history of web pages visited:

CODE:

```
#include <stdio.h>

#include <string.h>


#define MAX 10


char history[MAX][100];

int top = -1;


void visitPage()

{

   char page[100];


   if (top == MAX - 1)

   {
```

```
        printf("History is full!\n");
    }
    else
    {
        printf("Enter webpage URL: ");
        scanf("%s", page);

        top++;
        strcpy(history[top], page);

        printf("Visited: %s\n", page);
    }
}

void goBack()
{
    if (top == -1)
    {
        printf("No previous page available.\n");
    }
    else
    {
        printf("Going back from: %s\n", history[top]);
        top--;

        if (top == -1)
            printf("No previous page available.\n");
        else
```

```
        printf("Current page: %s\n", history[top]);
    }
}

void displayHistory()
{
    int i;

    if (top == -1)
    {
        printf("History is empty.\n");
    }
    else
    {
        printf("\nBrowser History:\n");

        for (i = top; i >= 0; i--)
        {
            printf("%s\n", history[i]);
        }
    }
}

int main()
{
    int choice;

    while (1)
```

```
{
    printf("\n--- BROWSER HISTORY ---\n");
    printf("1. Visit Page\n");
    printf("2. Go Back\n");
    printf("3. Display History\n");
    printf("4. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            visitPage();
            break;

        case 2:
            goBack();
            break;

        case 3:
            displayHistory();
            break;

        case 4:
            return 0;

        default:
```

```
            printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

OUTPUT:

```
 --- BROWSER HISTORY ---
1. Visit Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 1
Enter webpage URL: WHATSAPP
Visited: WHATSAPP

--- BROWSER HISTORY ---
1. Visit Page
2. Go Back
3. Display History
4. Exit
Enter your choice: 1
Enter webpage URL: chatgpt
Visited: chatgpt

--- BROWSER HISTORY ---
1. Visit Page
2. Go Back
```

3. Display History

4. Exit

Enter your choice: 2

Going back from: chatgpt

Current page: WHATSAPP

--- BROWSER HISTORY ---

1. Visit Page

2. Go Back

3. Display History

4. Exit

Enter your choice: 3

Browser History:

WHATSAPP

--- BROWSER HISTORY ---

1. Visit Page

2. Go Back

3. Display History

4. Exit

Enter your choice: 4

[Process completed - press Enter]

4)write a c program to sort a stack using temporary atack:

CODE:

#include <stdio.h>

```
#include <stdlib.h>

#define MAX 100

int stack[MAX];
int top = -1;

/* Push operation */
void push(int value)
{
   if (top == MAX - 1)
      printf("Stack Overflow\n");
   else
      stack[++top] = value;
}

/* Pop operation */
int pop()
{
   if (top == -1)
      return -1;

   return stack[top--];
}

/* Display stack */
void display()
{
```

```
    int i;

    for (i = top; i >= 0; i--)
        printf("%d ", stack[i]);

    printf("\n");
}

/* Sort stack in ascending order */
void sortStack()
{
    int temp[MAX];
    int tempTop = -1;
    int x;

    while (top != -1)
    {
        x = pop();

        while (tempTop != -1 && temp[tempTop] > x)
        {
            push(temp[tempTop--]);
        }

        temp[++tempTop] = x;
    }

    while (tempTop != -1)
```

```
    {
        push(temp[tempTop--]);
    }
}

int main()
{
    int n, i, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%d", &value);
        push(value);
    }

    printf("Original stack: ");
    display();

    sortStack();

    printf("Sorted stack: ");
    display();
```

```
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements:
12
44
23
55
65
Original stack: 65 55 23 44 12
Sorted stack: 12 23 44 55 65
```

5) The stock span problem is a financial problem where we have a series of daily price for a given stock denoted by an array and the task is to calculate the span of the stock's price for all days. The span of a stock's price for the day gives the maximum number of consecutive days leading up to the ith day (including current day) where stock price is less than or equal to its price on day i. Implement using stack.

CODE:

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
    stack[++top] = value;
```

```
}

int pop()
{
   return stack[top--];
}

int peek()
{
   return stack[top];
}

int isEmpty()
{
   return top == -1;
}

int main()
{
   int price[MAX], span[MAX];
   int n, i;

   printf("Enter number of days: ");
   scanf("%d", &n);

   printf("Enter stock prices:\n");
   for (i = 0; i < n; i++)
   {
```

```
        scanf("%d", &price[i]);
    }

    /* Calculate stock span */
    for (i = 0; i < n; i++)
    {
        while (!isEmpty() && price[peek()] <= price[i])
        {
            pop();
        }

        if (isEmpty())
            span[i] = i + 1;
        else
            span[i] = i - peek();

        push(i);
    }

    printf("Stock Span:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", span[i]);
    }

    return 0;
}
```

OUTPUT:

Enter number of days: 4

Enter stock prices:

10 30 20 19

Stock Span:

1 2 1 1

6) Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, return -1. Implement using stack.

CODE:

```
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int value)
{
   stack[++top] = value;
}

int pop()
{
   return stack[top--];
}

int peek()
{
```

```
    return stack[top];
}

int isEmpty()
{
    return top == -1;
}

int main()
{
    int a[MAX], freq[MAX], result[MAX];
    int n, i, j;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    /* Find frequency of each element */
    for (i = 0; i < n; i++)
    {
        freq[i] = 0;

        for (j = 0; j < n; j++)
```

```
    {
        if (a[i] == a[j])
            freq[i]++;
    }
}

/*
   Traverse from right to left.
   Stack stores indices of possible answers.
*/
for (i = n - 1; i >= 0; i--)
{
    while (!isEmpty() && freq[peek()] <= freq[i])
    {
        pop();
    }

    if (isEmpty())
        result[i] = -1;
    else
        result[i] = a[peek()];

    push(i);
}

printf("Output:\n");
for (i = 0; i < n; i++)
{
```

```
        printf("%d ", result[i]);
    }

    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter elements:

66

44

33

55

22

Output:

-1 -1 -1 -1 -1

7) Given a pattern containing only I's and D's — I for increasing and D for decreasing, devise an algorithm using stack to print the minimum number following that pattern. Digits from 1–9 and digits can't repeat.

CODE:

```
#include <stdio.h>

#define MAX 10

int stack[MAX];
int top = -1;

void push(int value)
{
```

```
    stack[++top] = value;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char pattern[MAX];
    int i, n, num;

    printf("Enter pattern (I/D): ");
    scanf("%s", pattern);

    n = 0;
    while (pattern[n] != '\0')
        n++;

    printf("Minimum number: ");

    num = 1;

    for (i = 0; i <= n; i++)
    {
        push(num);
        num++;
```

```
        if (i == n || pattern[i] == 'I')
        {
            while (top != -1)
            {
                printf("%d", pop());
            }
        }
    }

    return 0;
}
```

OUTPUT:

Enter pattern (I/D): 10

Minimum number: 321

8) Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from the digits of the array. All digits of the given array must be used to form the two numbers. Use a Queue for implementation.

CODE:

```
#include <stdio.h>

int main()
{
    int a[100], n, i, j, temp;
    int num1 = 0, num2 = 0;

    printf("Enter number of digits: ");
    scanf("%d", &n);
```

```
printf("Enter the digits:\n");
for(i = 0; i < n; i++)
{
   scanf("%d", &a[i]);
}

// Sort the digits in ascending order
for(i = 0; i < n - 1; i++)
{
   for(j = i + 1; j < n; j++)
   {
      if(a[i] > a[j])
      {
         temp = a[i];
         a[i] = a[j];
         a[j] = temp;
      }
   }
}

// Form two numbers
for(i = 0; i < n; i++)
{
   if(i % 2 == 0)
      num1 = num1 * 10 + a[i];
   else
      num2 = num2 * 10 + a[i];
```

```
    }

    printf("First number = %d\n", num1);
    printf("Second number = %d\n", num2);
    printf("Minimum sum = %d\n", num1 + num2);

    return 0;
}
```

OUTPUT:

```
Enter number of digits: 5
Enter the digits:
5
4
3
2
1
First number = 135
Second number = 24
Minimum sum = 159
```

9) Given an array and a positive integer K, find the first negative integer for each window (contiguous subarray) of size K. If a window does not contain a negative integer, print 0. Use a window / queue approach for implementation.

CODE:

```
#include <stdio.h>

int main()
{
    int a[100], n, k;
    int i, j, found;
```

```
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter the elements:\n");
for(i = 0; i < n; i++)
{
   scanf("%d", &a[i]);
}

printf("Enter window size K: ");
scanf("%d", &k);

printf("First negative integer in each window:\n");

for(i = 0; i <= n - k; i++)
{
   found = 0;

   for(j = i; j < i + k; j++)
   {
      if(a[j] < 0)
      {
         printf("%d ", a[j]);
         found = 1;
         break;
      }
   }
```

```
        if(found == 0)
        {
            printf("0 ");
        }
    }

    return 0;
}
```

OUTPUT:

Enter number of elements: 5

Enter the elements:

6

4

73

2

4

Enter window size K: 3

First negative integer in each window:

0 0 0

10) Given a matrix of dimensions m × n where each cell can have values 0, 1, or 2, where:

0 → Empty cell

1 → Cell having fresh orange

2 → Cell having rotten orange

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i, j) can rot fresh oranges in its neighboring cells (up, down, left and right). If impossible to rot every orange, return -1. Use a Queue for implementation.

CODE:

```
#include <stdio.h>

#define MAX 100

struct Queue
{
   int row[MAX];
   int col[MAX];
   int front;
   int rear;
};

void enqueue(struct Queue *q, int r, int c)
{
   q->rear++;
   q->row[q->rear] = r;
   q->col[q->rear] = c;
}

void dequeue(struct Queue *q, int *r, int *c)
{
   *r = q->row[q->front];
   *c = q->col[q->front];
   q->front++;
}

int main()
{
```

```
int a[10][10];
int m, n;
int i, j;
int fresh = 0;
int time = 0;
int r, c, nr, nc;
int size;

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

struct Queue q;
q.front = 0;
q.rear = -1;

printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter the matrix:\n");

for(i = 0; i < m; i++)
{
   for(j = 0; j < n; j++)
   {
      scanf("%d", &a[i][j]);

      if(a[i][j] == 1)
         fresh++;
```

```
        if(a[i][j] == 2)
            enqueue(&q, i, j);
    }
}

while(q.front <= q.rear && fresh > 0)
{
    size = q.rear - q.front + 1;

    for(i = 0; i < size; i++)
    {
        dequeue(&q, &r, &c);

        for(j = 0; j < 4; j++)
        {
            nr = r + dr[j];
            nc = c + dc[j];

            if(nr >= 0 && nr < m &&
               nc >= 0 && nc < n &&
               a[nr][nc] == 1)
            {
                a[nr][nc] = 2;
                fresh--;

                enqueue(&q, nr, nc);
            }
```

```
            }
        }

        time++;
    }

    if(fresh == 0)
        printf("Minimum time = %d\n", time);
    else
        printf("-1\n");

    return 0;
}
```

OUTPUT:

```
Enter rows and columns: 2 3
Enter the matrix:
10
20
30
45
32
21
Minimum time = 0
```